

UNIVERSITÀ DEGLI STUDI DI SALERNO

Corso di Laurea in Informatica

Corso di Ingegneria del Software

Anno Accademico 2024/2025

Tech4All

System Design Document

Piattaforma web per l'alfabetizzazione digitale

GRUPPO C06

Project manager

Ferdinando Boccia

Domenico D'Antuono

Team member

Marco Capuano

Giovanni Cerchia

Arcangelo Ciaramella

Silvana De Martino

Giovanni Esposito

Luigi Nasta

Giovanni Salsano

Giuseppe Staiano

Documento	SDD
-----------	-----

Versione	1.0
----------	-----

Indice

1	Introduzione	1
1.1	Scopo del documento	1
1.2	Obiettivi di progettazione	1
1.3	Compromessi	2
1.4	Definizioni e riferimenti	3
2	Architettura corrente	4
2.1	Vincoli che il sistema non eredita	4
2.2	Vincoli che il sistema eredita comunque	4
3	Architettura proposta	6
3.1	Panoramica	6
3.2	Decomposizione in sottosistemi	6
3.3	Mappatura hardware/software	8
3.4	Gestione dei dati persistenti	9
3.4.1	Scelta della tecnologia	9
3.4.2	Schema logico	10
3.4.3	Vincoli affidati al database	10
3.4.4	Transazioni	10
3.5	Controllo degli accessi e sicurezza	12
3.5.1	Riconoscimento del chiamante	12
3.5.2	Matrice di controllo degli accessi	13
3.5.3	Provenienza dell'identità	14
3.5.4	Altre misure	14
3.6	Controllo del flusso globale	15
3.7	Condizioni limite	15
3.7.1	Avvio	15
3.7.2	Spegnimento	15
3.7.3	Fallimenti	16
4	Servizi dei sottosistemi	17
4.1	Autenticazione	17
4.2	Gestione account	18
4.3	Gestione tutorial	18
4.4	Gestione quiz	19
4.5	Gestione feedback	20

4.6	Gestione obiettivi	21
5	Progettazione degli oggetti	23
5.1	Compromessi di progettazione degli oggetti	23
5.2	Linee guida di documentazione delle interfacce	24
5.3	Organizzazione in package	24
5.4	Contratti delle classi principali	25
5.4.1	Livello di persistenza	25
5.4.2	Serializzazione	26
5.4.3	Gestione degli errori	27
5.4.4	Iniezione delle dipendenze	27
5.5	Pattern di progettazione adottati	28
5.5.1	Data Access Object	28
5.5.2	Facade (lato client)	28
5.5.3	Chain of Responsibility (middleware)	28
5.5.4	Template Method (transazioni)	28
5.6	Pattern valutati e non adottati	29
6	Decisioni di progetto e razionale	30
6.1	DP_1 — La sessione è un token firmato in un cookie httpOnly	30
6.2	DP_2 — La correzione del quiz avviene solo sul server	30
6.3	DP_3 — Le risposte sono associate per identificativo	31
6.4	DP_4 — Il conteggio dei quiz superati è derivato	31
6.5	DP_5 — La valutazione media è mantenuta dal database	31
6.6	DP_6 — Quiz, domande e risposte formano un solo aggregato	32
6.7	DP_7 — Il client non condivide codice con il server	32
6.8	DP_8 — L'assistente conversazionale è un componente esterno	32
7	Glossario	33
7.1	Termini del dominio	33
7.2	Termini tecnici	34
7.3	Acronimi	35
A	Tracciabilità delle decisioni di progetto	36
A.1	Requisiti non funzionali, obiettivi e meccanismi	36
A.2	Casi d'uso e sottosistemi	37
	Riferimenti bibliografici	39

Introduzione

1.1 Scopo del documento

Questo documento descrive come Tech4All è costruito. Muove dai requisiti non funzionali del RAD per ricavarne gli obiettivi di progettazione, decompone il sistema in sottosistemi, ne definisce l'interfaccia, e scende fino alla progettazione delle classi che li realizzano.

Rispetto all'impostazione di Bruegge e Dutoit [1] il documento accorpa il System Design Document e l'Object Design Document. La scelta è motivata dalla scala del sistema: un ODD autonomo, per un progetto di queste dimensioni, sarebbe in gran parte frontespizio, glossario e ripetizione delle premesse. L'object design occupa quindi il capitolo 5, con tutti i contenuti che il testo di riferimento gli attribuisce.

1.2 Obiettivi di progettazione

Gli obiettivi di progettazione (*design goal*) traducono i requisiti non funzionali in criteri che guidano le decisioni tecniche. Ciascuno è derivato da uno o più requisiti del RAD; la corrispondenza completa è nell'appendice A.

Id	Obiettivo	Significato operativo
DG_1	Fiducia nel confine	Ciò che arriva dall'esterno non è mai considerato attendibile: né i dati, né l'identità dichiarata, né i nomi dei file. Il sistema decide sulla base di ciò che possiede, non di ciò che gli viene detto.
DG_2	Un solo luogo per ogni decisione	Ogni regola — un vincolo di dominio, una soglia, un permesso — è definita in un punto solo. Regole duplicate divergono.
DG_3	Verificabilità	Ogni componente deve poter essere esercitato in isolamento, senza database né rete. Ciò che non si riesce a provare facilmente è, di norma, mal progettato.

Id	Obiettivo	Significato operativo
DG_4	Uniformità dei sottosistemi	I sottosistemi hanno la stessa struttura interna e le stesse convenzioni. Chi ne conosce uno sa orientarsi negli altri.
DG_5	Coerenza dei dati	Le operazioni che toccano più entità sono atomiche; gli invarianti che il database può garantire sono affidati al database.
DG_6	Costo di esercizio contenuto	Nessuna dipendenza da servizi a pagamento, nessuna infrastruttura oltre un processo applicativo e un DBMS.

1.3 Compromessi

Gli obiettivi di progettazione entrano in conflitto fra loro. I compromessi che seguono sono le decisioni prese quando non era possibile soddisfarli tutti, con la ragione della scelta.

Compromesso	Alternativa scartata	Motivazione
Sicurezza contro immediatezza della risposta	Correggere il quiz nel browser, evitando un giro di rete	La correzione locale renderebbe le soluzioni visibili prima della consegna, annullando il valore della verifica (OB_6). Il costo è una richiesta HTTP: trascurabile rispetto a ciò che si guadagna.
Uniformità contro concisione	Servizi ridotti a semplici passacarte verso i DAO	Alcuni servizi si limitano a delegare, e potrebbero essere eliminati. Sono stati mantenuti perché la loro assenza renderebbe i sottosistemi diversi fra loro senza un motivo di dominio (DG_4).
Coerenza contro indipendenza dal DBMS	Calcolare la valutazione media nel codice applicativo	La media è mantenuta da trigger. Il valore resta corretto anche di fronte a scritture concorrenti, al prezzo di una logica che vive nel database e non è verificata dai test unitari (DG_5).
Semplicità contro completezza funzionale	Recupero della password via email	Realizzarlo richiederebbe un servizio di posta transazionale e un dominio verificato, contro DG_6. Il sistema rinuncia alla funzionalità e lo dichiara apertamente.

Compromesso	Alternativa scartata	Motivazione
Verificabilità contro fedeltà dei test	Test di integrazione su un database reale	La suite sostituisce il livello di persistenza con dei doppi: gira ovunque in pochi secondi (DG_3), ma non verifica le query SQL, che restano coperte solo dalle prove manuali di sistema.

L'ultimo compromesso è quello con il rischio residuo più alto ed è esplicitamente dichiarato anche nel Test Document: un errore in una query verrebbe intercettato solo all'esecuzione. È stato accettato perché il livello di persistenza è sottile e privo di logica condizionale, mentre il costo di un database nella catena di integrazione continua sarebbe permanente.

1.4 Definizioni e riferimenti

I termini del dominio e quelli tecnici sono definiti nel capitolo 7, condiviso con gli altri documenti del progetto. Il documento presuppone la lettura del RAD, di cui riprende identificativi di requisiti e casi d'uso senza ridefinirli.

Architettura corrente

Tech4All non sostituisce alcun sistema software esistente. La prassi che intende migliorare, descritta nel RAD, non è mediata da alcun sistema informatico: è un insieme di procedure umane — la delega a un conoscente, la ricerca autonoma in rete, il ricorso a un servizio di assistenza — che non ha un'architettura da documentare.

L'assenza di un antecedente ha conseguenze precise sulla progettazione, ed è per questo che il capitolo esiste anziché essere omissso.

2.1 Vincoli che il sistema non eredita

Vincolo assente	Conseguenza sulla progettazione
Interfacce da preservare	Nessun formato di scambio, protocollo o schema è imposto dall'esterno. La decomposizione in sottosistemi e la forma dell'interfaccia HTTP possono essere scelte in funzione del dominio e non della compatibilità.
Dati da migrare	Non esiste una base di dati preesistente. Lo schema può essere normalizzato liberamente e gli invarianti possono essere affidati al DBMS senza dover accogliere dati che li violano.
Tecnologie imposte	Nessuna piattaforma è vincolata da scelte pregresse. I vincoli sulla tecnologia sono solo quelli di progetto (V_1–V_5 del RAD).
Utenti da riconvertire	Non esistono abitudini d'uso consolidate su un sistema precedente. L'unico riferimento è la competenza digitale del pubblico, che è bassa per definizione: è questo, e non la continuità con un sistema esistente, a guidare le scelte di interazione.

2.2 Vincoli che il sistema eredita comunque

L'assenza di un sistema precedente non significa assenza di vincoli. Tre condizioni dell'ambiente sono date e la progettazione deve accoglierle.

Il browser

è l'unico canale d'accesso previsto (V_1). Il sistema non può presupporre installazioni sul dispositivo dell'utente né capacità di calcolo particolari, e deve funzionare su dispositivi datati.

La connessione

è intermittente e di qualità variabile. Il sistema è progettato perché ogni operazione sia autonoma: nessuna funzionalità richiede una sessione di lavoro continua o il completamento di più passaggi in sequenza entro un limite di tempo.

La competenza dell'utente

è il vincolo più stringente. Determina la scelta di rendere pubblica la consultazione (OB_1), l'obbligo di messaggi d'errore comprensibili (RNF_U_2) e la richiesta di conferma sulle operazioni irreversibili (RNF_U_3).

Il capitolo seguente descrive l'architettura proposta a partire da queste condizioni.

Architettura proposta

3.1 Panoramica

Tech4All è organizzato come un'applicazione client/server. Il client è un'applicazione Next.js che gira nel browser; il server è un processo Node.js che espone un'interfaccia HTTP con rappresentazioni JSON e conserva i dati in un DBMS relazionale. Le due parti comunicano solo attraverso quell'interfaccia: il client non condivide codice con il server e non ne conosce il modello a oggetti.

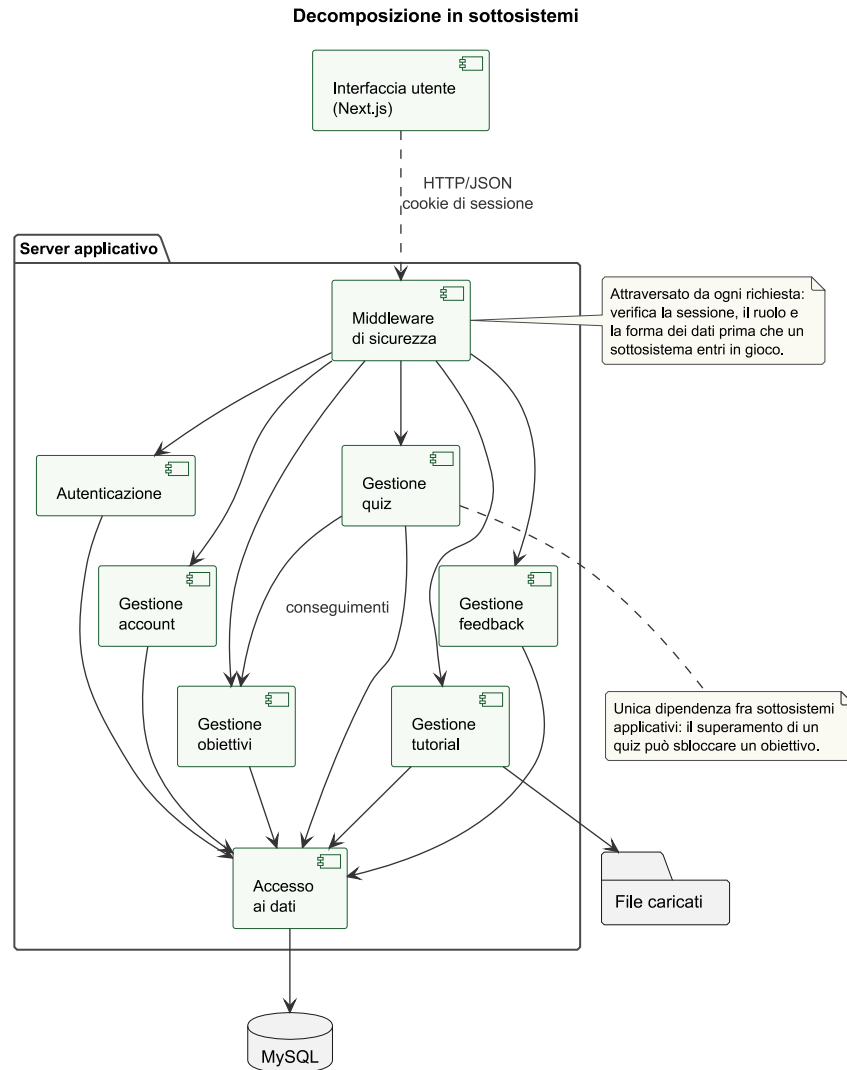
Lo stile architetturale è a livelli, replicato identico dentro ogni sottosistema (DG_4):

- | | |
|----------------|--|
| Rotte | traducono una richiesta HTTP in una chiamata a un servizio. Verificano la forma dei dati, applicano il controllo degli accessi e serializzano la risposta. Non contengono regole di dominio. |
| Servizi | realizzano le regole di dominio, coordinano più DAO e delimitano le transazioni. Non conoscono HTTP. |
| DAO | traducono fra entità e tabelle. Non contengono decisioni. |
| Entità | rappresentano i concetti del dominio e ne custodiscono lo stato. Non dipendono da nulla. |

La direzione delle dipendenze è sempre la stessa: ogni livello conosce quello sottostante e ignora quello soprastante.

3.2 Decomposizione in sottosistemi

Il sistema è decomposto in sei sottosistemi applicativi, in corrispondenza diretta con le aree funzionali del RAD, più due sottosistemi di infrastruttura attraversati da tutti gli altri.



Sottosistema	Responsabilità
Autenticazione	Registrazione, riconoscimento delle credenziali, apertura e chiusura della sessione. Non decide chi può fare cosa: stabilisce soltanto chi è il chiamante.
Gestione account	Consultazione e modifica dei dati anagrafici, cambio password, cancellazione dell'account, elenco degli utenti per l'amministratore.
Gestione tutorial	Catalogo, ricerca, filtro, pubblicazione, aggiornamento e ritiro dei contenuti; sanificazione dell'HTML e normalizzazione delle immagini.
Gestione quiz	Definizione dei quiz, erogazione priva di soluzioni, correzione e registrazione degli svolgimenti.
Gestione feedback	Inserimento, modifica, rimozione e moderazione delle valutazioni.

Sottosistema	Responsabilità
Gestione obiettivi	Definizione degli obiettivi e assegnazione automatica dei conseguimenti.
Middleware di sicurezza	Verifica del token di sessione, controllo del ruolo, validazione della forma delle richieste, gestione degli errori e degli upload.
Accesso ai dati	Connessioni al DBMS, esecuzione delle interrogazioni, delimitazione delle transazioni.

L'accoppiamento fra sottosistemi applicativi è quasi nullo: l'unica dipendenza è quella di *Gestione quiz* verso *Gestione obiettivi*, che riflette la relazione di estensione fra UC_09 e UC_10. Non è una dipendenza accidentale ma la traduzione di un fatto di dominio: il conseguimento di un badge è una conseguenza del superamento di un quiz.

3.3 Mappatura hardware/software

Il sistema è distribuito su tre nodi logici: il dispositivo dell'utente, il nodo applicativo e il nodo dati. In sviluppo i due nodi server coincidono; la separazione è logica e non impone una topologia.

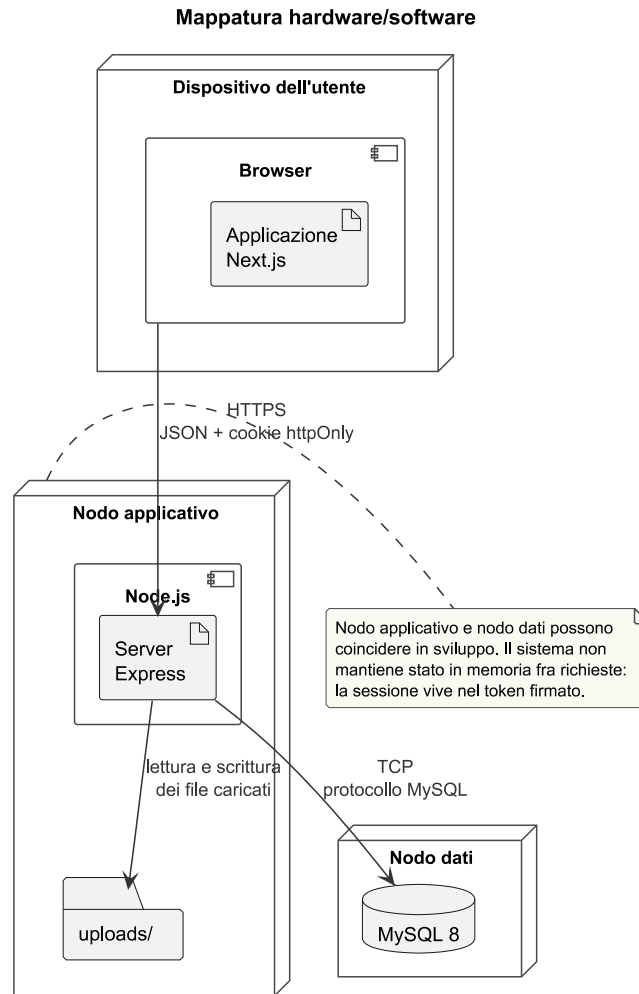


Figura 3.2. Mappatura hardware/software.

Il server non conserva stato in memoria fra una richiesta e l'altra: la sessione vive interamente nel token firmato che il browser riallega a ogni richiesta. È la proprietà che rende il nodo applicativo replicabile senza alcuna condivisione di stato, se in futuro servisse.

I file caricati risiedono sul filesystem del nodo applicativo e sono serviti staticamente. È la scelta più semplice compatibile con DG_6; il prezzo è che, in presenza di più nodi applicativi, quella cartella dovrebbe essere condivisa.

3.4 Gestione dei dati persistenti

3.4.1 Scelta della tecnologia

La persistenza è affidata a MySQL 8. La scelta discende dalla natura dei dati, non da preferenze: il dominio è fatto di entità con relazioni fisse e molteplicità note, e gli invarianti più importanti — l'unicità di un feedback per coppia utente-tutorial, l'esistenza di un solo quiz per tutorial, la rimozione a cascata di ciò che dipende da un tutorial — sono esprimibili come vincoli del modello relazionale. Affidarli al

DBMS significa che restano veri anche se il codice applicativo sbaglia (DG_5).

3.4.2 Schema logico

Le tre tabelle `feedback`, `svolgimento` e `conseguimento` hanno chiave primaria composta dalle due chiavi esterne che collegano: è la traduzione diretta delle classi di associazione del modello di analisi, e rende impossibile per costruzione l'esistenza di un duplicato.

3.4.3 Vincoli affidati al database

Vincolo	Realizzazione
Un utente lascia al più un feedback per tutorial	Chiave primaria (<code>utente_id</code> , <code>tutorial_id</code>) su <code>feedback</code>
Un tutorial ha al più un quiz	Vincolo di unicità su <code>quiz.tutorial_id</code>
Un utente ha al più uno svolgimento per quiz	Chiave primaria (<code>utente_id</code> , <code>quiz_id</code>) su <code>svolgimento</code>
Rimuovere un tutorial rimuove quiz, domande, risposte, svolgimenti e feedback	Vincoli di integrità referenziale con <code>ON DELETE CASCADE</code>
La valutazione è compresa fra 1 e 5	Vincolo <code>CHECK</code> sulle tabelle <code>feedback</code> e <code>tutorial</code>
Il ruolo assume solo i valori previsti	Tipo <code>ENUM</code> su <code>utente.ruolo</code>
La valutazione media riflette i feedback ricevuti	Tre trigger su <code>feedback</code> che ricalcolano <code>tutorial.valutazione</code>

3.4.4 Transazioni

Due operazioni coinvolgono più tabelle e devono essere atomiche:

- la **definizione di un quiz**, che inserisce il quiz, le sue domande e le risposte di ciascuna domanda;
- la **correzione di un quiz**, che registra lo svolgimento, ricalcola il numero di quiz superati e assegna gli eventuali conseguimenti.

Entrambe sono racchiuse nella funzione `withTransaction`, che apre la transazione, esegue il lavoro, effettua il commit e in caso di eccezione il rollback, rilasciando sempre la connessione. I DAO coinvolti accettano la connessione della transazione come parametro: usare il pool comune all'interno di una transazione già aperta porterebbe la stessa richiesta ad attendere i lock che essa stessa detiene.

Schema logico della base di dati

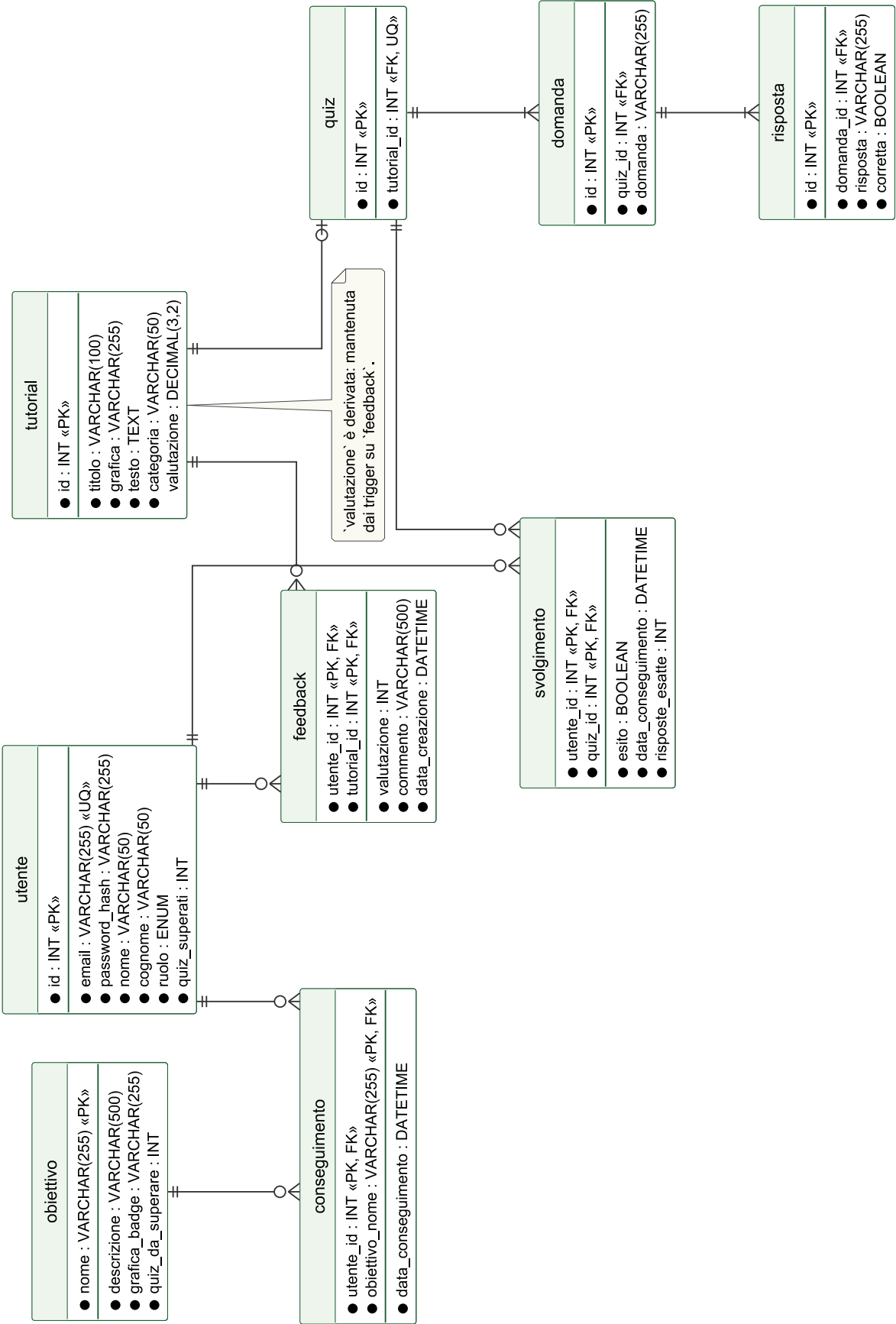


Figura 3.3. Schema logico della base di dati.

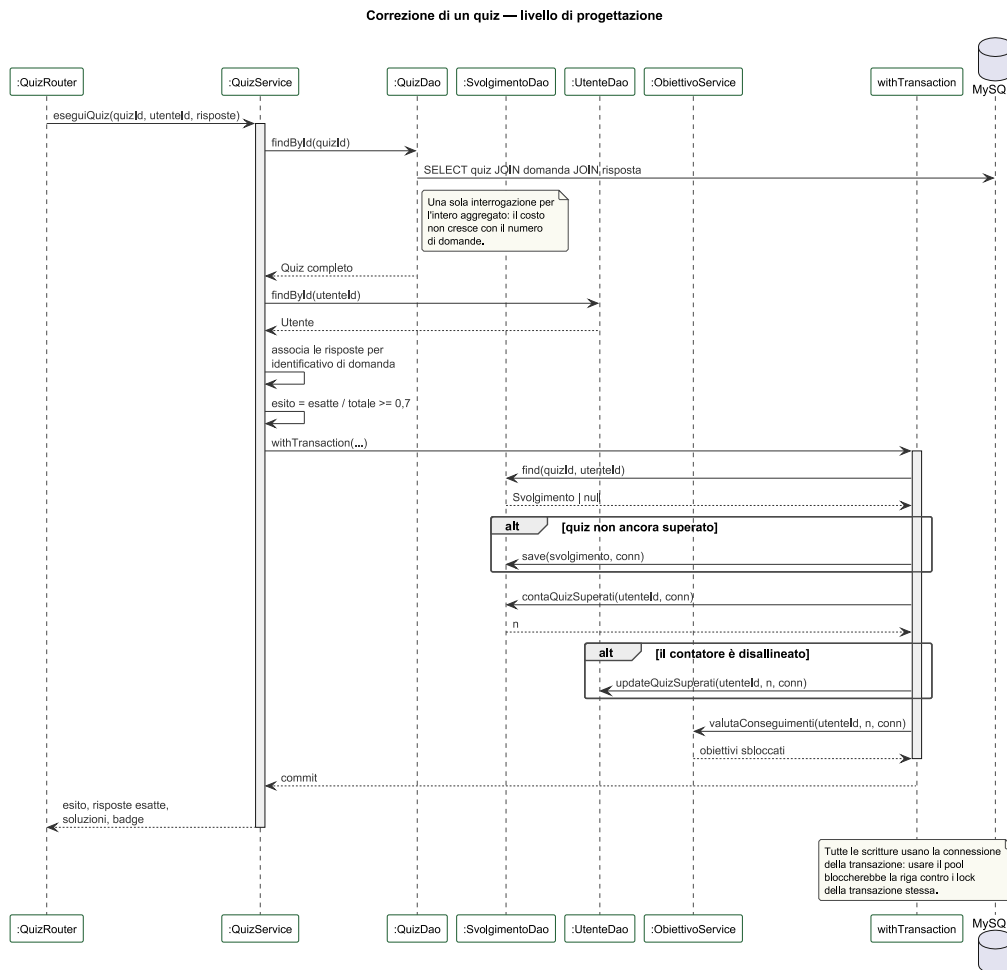


Figura 3.4. Correzione di un quiz: propagazione della connessione transazionale ai DAO coinvolti.

3.5 Controllo degli accessi e sicurezza

3.5.1 Riconoscimento del chiamante

La sessione è un token JWT [3] firmato dal server, contenente l'identificativo dell'utente e il suo ruolo, depositato in un cookie `httpOnly`, `SameSite=Lax` e, in produzione, `Secure`. Il token non è leggibile dal JavaScript di pagina: una vulnerabilità di scripting non permette di esfiltrarlo.

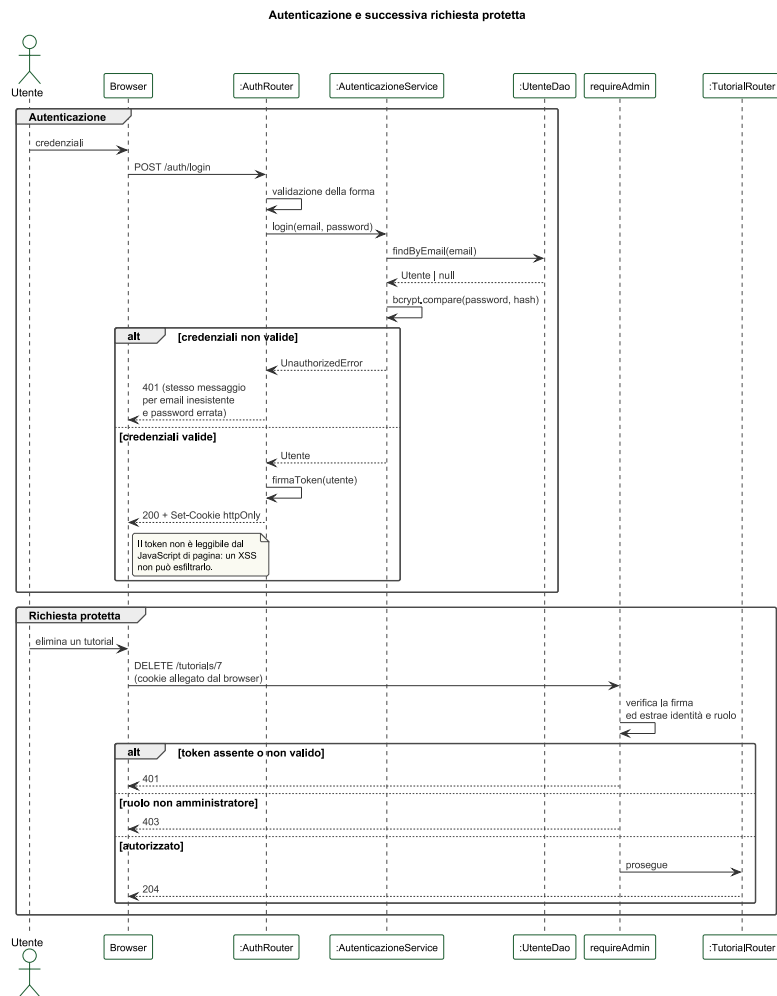


Figura 3.5. Autenticazione e successiva richiesta protetta.

3.5.2 Matrice di controllo degli accessi

Il controllo è realizzato da due middleware: `requireAuth` rifiuta le richieste prive di una sessione valida; `requireAdmin` rifiuta in più quelle di chi non ha ruolo amministratore. Sono attraversati prima che il sottosistema applicativo entri in gioco.

Operazione	Visitatore	Utente	Amm.
Registrarsi, autenticarsi	✓	—	—
Consultare catalogo, tutorial, feedback, quiz, obiettivi	✓	✓	✓
Consultare e modificare il proprio profilo	—	✓	✓
Eliminare il proprio account	—	✓	✓
Consegnare un quiz	—	✓	✓

Operazione	Visitatore	Utente	Amm.
Gestire il proprio feedback	—	✓	✓
Consultare i propri badge	—	✓	✓
Pubblicare, aggiornare, ritirare tutorial	—	—	✓
Definire e rimuovere quiz	—	—	✓
Caricare ed eliminare immagini	—	—	✓
Definire, aggiornare, eliminare obiettivi	—	—	✓
Consultare ed eliminare account altrui	—	—	✓
Moderare i feedback altrui	—	—	✓

3.5.3 Provenienza dell'identità

Un principio attraversa tutto il sistema (DG_1): l'autore di un'operazione è sempre l'utente della sessione. Nessuna rotta accetta un identificativo utente dal corpo o dai parametri della richiesta per stabilire per conto di chi agire. L'unico caso in cui un identificativo altrui compare in una rotta è la moderazione dei feedback, dove indica il *bersaglio* dell'operazione e non il suo autore, ed è accessibile solo agli amministratori.

3.5.4 Altre misure

Minaccia	Contromisura
Furto di credenziali dal database	Le password sono conservate come hash bcrypt [5], con un fattore di costo configurabile.
Enumerazione degli account	Credenziali errate ed email inesistente producono la stessa risposta.
Iniezione di codice nei contenuti	L'HTML dei tutorial è sanificato in ingresso contro una whitelist di marcatori e attributi; schemi di URL diversi da http, https e mailto sono rimossi [4].
Iniezione SQL	Tutte le interrogazioni usano parametri preparati; nessun valore proveniente dall'esterno è concatenato nel testo di una query.
Traversamento del filesystem	I nomi dei file caricati sono generati dal server; l'eliminazione accetta solo nomi semplici, risolti e verificati dentro la cartella consentita.
Richieste da origini non previste	Il CORS autorizza una sola origine, configurata, e ammette le credenziali solo per essa.

Minaccia	Contromisura
Divulgazione di dettagli interni	Gli errori imprevisi producono una risposta generica; i dettagli restano nei log del server.

3.6 Controllo del flusso globale

Il sistema è guidato dagli eventi: non esiste un flusso di controllo proprio, ma una sequenza di reazioni a richieste HTTP. Ogni richiesta attraversa sempre le stesse fasi, nello stesso ordine.

1. **Ingresso:** parsing del corpo e dei cookie, verifica dell'origine.
2. **Autenticazione:** lettura e verifica del token, se la rotta la richiede.
3. **Autorizzazione:** verifica del ruolo, se la rotta la richiede.
4. **Validazione:** controllo della forma dei dati; una violazione interrompe qui, senza raggiungere il servizio.
5. **Dominio:** il servizio applica le regole e coordina i DAO.
6. **Serializzazione:** l'entità è tradotta in DTO.
7. **Uscita:** la risposta è emessa; se una fase ha sollevato un errore, il gestore centralizzato lo traduce in uno stato HTTP.

Le fasi 2, 3, 4 e 7 sono realizzate da middleware e sono quindi identiche per tutti i sottosistemi (DG_2, DG_4). Un solo processo serve tutte le richieste; la concorrenza è quella del ciclo di eventi di Node.js, e non esiste stato condiviso in memoria fra richieste diverse.

3.7 Condizioni limite

3.7.1 Avvio

All'avvio il server verifica la presenza di tutte le variabili d'ambiente obbligatorie e fallisce immediatamente elencando quelle mancanti, prima di mettersi in ascolto. È una scelta deliberata: un sistema che parte senza il segreto di firma o senza le credenziali del database fallirebbe più tardi, in modo meno comprensibile.

Il pool di connessioni è creato al primo utilizzo e non apre connessioni all'avvio: l'indisponibilità del database non impedisce al processo di partire, ma si manifesta come errore sulle richieste che ne hanno bisogno.

3.7.2 Spegnimento

Alla ricezione di SIGINT o SIGTERM il server smette di accettare nuove connessioni, attende il completamento di quelle in corso, chiude il pool e termina. Le transazioni aperte al momento dell'interruzione

vengono annullate dal DBMS alla caduta della connessione: il database non resta in uno stato intermedio.

3.7.3 Fallimenti

Condizione	Comportamento del sistema
Database irraggiungibile	Le richieste che richiedono dati falliscono con 500 e un messaggio generico; le rotte statiche continuano a rispondere.
Errore durante una transazione	Rollback automatico: nessuna scrittura parziale resta applicata.
File di upload non scrivibile	La pubblicazione fallisce con un errore di validazione; il tutorial non viene creato.
Token scaduto o manomesso	Trattato come sessione assente: 401 sulle rotte protette, accesso anonimo su quelle pubbliche.
Richiesta con corpo eccessivo	Rifiutata dal limite di dimensione prima di raggiungere l'applicazione.
Eccezione non prevista	Intercettata dal gestore centralizzato, registrata nei log e tradotta in 500 privo di dettagli interni.

Servizi dei sottosistemi

Questo capitolo definisce l'interfaccia di ciascun sottosistema su due livelli: le risorse HTTP che espone verso il client e le operazioni che il suo servizio offre al resto del server. Il primo livello è il contratto esterno del sistema; il secondo è il contratto fra i livelli architetturali.

Convenzioni delle risposte. Le operazioni di lettura rispondono 200, le creazioni 201, le operazioni che non restituiscono contenuto 204. Gli errori seguono la gerarchia di sezione 5.4.3: 400 dati non validi, 401 sessione assente, 403 permessi insufficienti, 404 risorsa inesistente, 409 conflitto con lo stato attuale.

4.1 Autenticazione

Risorsa	Accesso	Descrizione
POST /auth/registrazione	pubblico	Crea un account con ruolo utente. Non apre la sessione.
POST /auth/login	pubblico	Verifica le credenziali e deposita il cookie di sessione.
POST /auth/logout	pubblico	Invalida il cookie di sessione.
GET /auth/me	autenticato	Profilo dell'utente della sessione corrente.
GET /auth/email-disponibile	pubblico	Indica se un'email è ancora libera.

Operazioni di AutenticazioneService

Operazione	Contratto
registra(dati)	Crea l'utente con la password sottoposta ad hashing. <i>Solleva</i> <code>ValidationError</code> se la password non rispetta la politica, <code>ConflictError</code> se l'email è già in uso.

Operazione	Contratto
<code>login(email, password)</code>	Restituisce l'utente se le credenziali sono valide. <i>Solleva</i> <code>UnauthorizedError</code> con lo stesso messaggio sia per email inesistente sia per password errata.
<code>emailEsiste(email)</code>	Restituisce un booleano. Non solleva eccezioni.

4.2 Gestione account

Risorsa	Accesso	Descrizione
<code>GET /accounts/me</code>	autenticato	Profilo dell'utente della sessione.
<code>PUT /accounts/me</code>	autenticato	Aggiorna dati anagrafici e password.
<code>DELETE /accounts/me</code>	autenticato	Elimina il proprio account.
<code>GET /accounts/me/badge</code>	autenticato	Badge conseguiti.
<code>GET /accounts</code>	amministratore	Elenco degli utenti registrati.
<code>DELETE /accounts/:id</code>	amministratore	Elimina l'account indicato.

Operazioni di AccountService

Operazione	Contratto
<code>getUtente(id)</code>	<i>Solleva</i> <code>NotFoundError</code> se l'utente non esiste.
<code>getUtenti()</code>	Elenco completo, ordinato per cognome e nome.
<code>aggiornaProfilo(id, dati)</code>	I campi assenti restano invariati. Il cambio password richiede quella attuale. <i>Solleva</i> <code>NotFoundError</code> , <code>ConflictError</code> se la nuova email è già in uso, <code>UnauthorizedError</code> se la password attuale non è corretta, <code>ValidationError</code> se la nuova password è debole.
<code>eliminaUtente(id)</code>	Rimuove l'account; feedback, svolgimenti e conseguimenti cadono per vincolo di integrità. <i>Solleva</i> <code>NotFoundError</code> .

4.3 Gestione tutorial

Risorsa	Accesso	Descrizione
GET /tutorials	pubblico	Catalogo, con filtro facoltativo per categoria.
GET /tutorials/categorie	pubblico	Categorie ammesse.
GET /tutorials/ricerca	pubblico	Ricerca per parola chiave.
GET /tutorials/:id	pubblico	Dettaglio di un tutorial.
POST /tutorials	amministratore	Pubblica un tutorial.
PUT /tutorials/:id	amministratore	Aggiorna un tutorial.
DELETE /tutorials/:id	amministratore	Ritira un tutorial.
POST /tutorials/immagini	amministratore	Carica un'immagine di contenuto.
DELETE /tutorials/immagini/:nome	amministratore	Elimina un'immagine di contenuto.

Operazioni di TutorialService

Operazione	Contratto
getTutorials(categoria?)	Senza argomento restituisce l'intero catalogo. <i>Solleva</i> <code>ValidationError</code> se la categoria non esiste.
getTutorial(id)	<i>Solleva</i> <code>NotFoundError</code> .
cercaTutorial(chiave)	Chiave vuota restituisce un elenco vuoto senza interrogare il database.
creaTutorial(dati)	Valida i vincoli di dominio, sanifica il contenuto e persiste. <i>Solleva</i> <code>ValidationError</code> .
aggiornaTutorial(id, dati)	Come sopra. <i>Solleva</i> anche <code>NotFoundError</code> .
eliminaTutorial(id)	Quiz, domande, risposte, svolgimenti e feedback cadono in cascata. <i>Solleva</i> <code>NotFoundError</code> .

4.4 Gestione quiz

Risorsa	Accesso	Descrizione
GET /quiz/tutorial/:tutorialId	pubblico	Quiz del tutorial, privo delle soluzioni.
POST /quiz/tutorial/:tutorialId	amministratore	Definisce il quiz.

Risorsa	Accesso	Descrizione
PUT /quiz/:id	amministratore	Sostituisce le domande.
DELETE /quiz/:id	amministratore	Rimuove il quiz.
POST /quiz/:id/svolgimenti	autenticato	Consegna le risposte e ottiene la correzione.

Operazioni di QuizService

Operazione	Contratto
<code>getQuizPerTutorial(tutorialId)</code>	<i>Solleva</i> <code>NotFoundError</code> se il tutorial non ha un quiz.
<code>creaQuiz(tutorialId, domande)</code>	Inserisce quiz, domande e risposte in un'unica transazione. <i>Solleva</i> <code>NotFoundError</code> se il tutorial non esiste, <code>ConflictError</code> se ha già un quiz, <code>ValidationError</code> se una domanda non ha esattamente una risposta corretta o viola i limiti di lunghezza.
<code>aggiornaQuiz(quizId, domande)</code>	Sostituisce integralmente le domande; gli svolgimenti sono conservati. <i>Solleva</i> <code>NotFoundError</code> , <code>ValidationError</code> .
<code>eliminaQuiz(quizId)</code>	<i>Solleva</i> <code>NotFoundError</code> .
<code>eseguiQuiz(quizId, utenteId, risposte)</code>	Corregge, registra lo svolgimento, ricalcola i quiz superati e delega l'assegnazione dei badge. Non declassa un quiz già superato. <i>Solleva</i> <code>NotFoundError</code> se quiz o utente non esistono, <code>ValidationError</code> se il quiz non ha domande.

4.5 Gestione feedback

Risorsa	Accesso	Descrizione
GET /feedback/tutorial/:tutorialId	pubblico	Feedback di un tutorial.
GET /feedback/me	autenticato	Feedback dell'utente.
POST /feedback/tutorial/:tutorialId	autenticato	Inserisce il proprio feedback.
PUT /feedback/tutorial/:tutorialId	autenticato	Modifica il proprio feedback.

Risorsa	Accesso	Descrizione
DELETE /feedback/tutorial/:tutorialId	autenticato	Elimina il proprio feedback.
DELETE /feedback/tutorial/:tutorialId/ utente/:utenteId	autenticato	Elimina il feedback indicato; consentito solo se ne è l'autore o è amministratore.

Operazioni di FeedbackService

Operazione	Contratto
getFeedbackTutorial(tutorialId)	Elenco ordinato per data decrescente.
getFeedbackUtente(utenteId)	Elenco dei feedback dell'utente.
creaFeedback(utenteId, tutorialId, dati)	<i>Solleva</i> <code>ValidationError</code> sui vincoli di valutazione e commento, <code>NotFoundError</code> se il tutorial non esiste, <code>ConflictError</code> se l'utente ha già valutato quel tutorial.
aggiornaFeedback(utenteId, tutorialId, dati)	<i>Solleva</i> <code>ValidationError</code> , <code>NotFoundError</code> .
eliminaFeedback(richiedente, autore, tutorialId, isAdmin)	Un utente che tenti di eliminare un feedback altrui riceve <code>NotFoundError</code> , non <code>ForbiddenError</code> : negare l'esistenza è preferibile a confermarla a chi non ha titolo.

4.6 Gestione obiettivi

Risorsa	Accesso	Descrizione
GET /obiettivi	pubblico	Obiettivi raggiungibili e relative soglie.
POST /obiettivi	amministratore	Definisce un obiettivo.
PUT /obiettivi/:nome	amministratore	Aggiorna un obiettivo.
DELETE /obiettivi/:nome	amministratore	Elimina un obiettivo.

Operazioni di ObiettivoService

Operazione	Contratto
getObiettivi()	Catalogo ordinato per soglia crescente.

Operazione	Contratto
<code>getBadgeUtente(utenteId)</code>	Conseguimenti abbinati al rispettivo obiettivo; i conseguimenti il cui obiettivo è stato eliminato sono omessi.
<code>valutaConseguimenti(utenteId, quizSuperati, conn?)</code>	Assegna gli obiettivi la cui soglia è raggiunta e non ancora conseguiti; restituisce quelli sbloccati dalla chiamata. È idempotente: invocata due volte con lo stesso stato non assegna nulla la seconda volta. Accetta una connessione transazionale.
<code>creaObiettivo(dati)</code>	<i>Solleva</i> <code>ValidationError</code> , <code>ConflictError</code> se il nome è già usato.
<code>aggiornaObiettivo(nome, dati)</code>	<i>Solleva</i> <code>NotFoundError</code> , <code>ValidationError</code> .
<code>eliminaObiettivo(nome)</code>	I conseguimenti associati cadono in cascata. <i>Solleva</i> <code>NotFoundError</code> .

Progettazione degli oggetti

Questo capitolo scende dal livello dei sottosistemi a quello delle classi. Fissa le convenzioni con cui il codice è scritto, descrive l'organizzazione in package, documenta i contratti delle classi principali e discute i pattern adottati — e quelli valutati e scartati.

5.1 Compromessi di progettazione degli oggetti

Compromesso	Alternativa scartata	Motivazione
Entità come classi con accessori	Strutture dati anonime, senza comportamento	Le classi permettono di collocare nel dominio le poche operazioni che gli appartengono davvero, come «la risposta corretta di questa domanda». Il prezzo è più codice di accesso.
Un DAO per aggregato	Un DAO per tabella	Domande e risposte non hanno ciclo di vita autonomo: separarle in DAO propri produrrebbe classi che nessuno può usare correttamente da sole e moltiplicherebbe le interrogazioni.
Iniezione tramite parametri con valore predefinito	Un contenitore di dipendenze	Il valore predefinito rende testabile ogni classe senza introdurre un framework e senza che il codice di produzione debba dichiarare un grafo di dipendenze.
Eccezioni tipizzate	Valori di ritorno che descrivono l'esito	Le eccezioni tolgono dalle rotte la propagazione manuale dell'errore, al prezzo di un percorso di controllo non visibile nella firma dei metodi.

5.2 Linee guida di documentazione delle interfacce

Le convenzioni seguenti valgono per tutto il codice del server e sono verificate, per quanto possibile, dall'analisi statica.

Lingua I concetti del dominio conservano il nome italiano (Utente, Tutorial, Svolgimento); i termini tecnici restano in inglese (Service, Dao, middleware, findById). La regola tiene allineati codice e glossario senza tradurre termini che non hanno un equivalente italiano consolidato.

Nomi Classi in PascalCase, metodi e variabili in camelCase, costanti in MAIUSCOLO_CON_UNDERSCORE, tabelle e colonne in snake_case. I metodi dei DAO usano i verbi find, create, update, delete; quelli dei servizi usano il verbo del dominio (creaQuiz, eseguiQuiz).

Visibilità

I campi delle entità sono privati e accessibili tramite metodi; è pubblico solo ciò che fa parte del contratto.

Documentazione

Ogni classe e ogni metodo pubblico hanno un commento che ne dichiara lo scopo, i parametri, il valore restituito e le eccezioni sollevate. I commenti spiegano *perché*, non *che cosa*: ciò che il codice già dice non viene ripetuto.

Tipi Nessun uso di any; il controllo dei tipi è attivo in modalità stretta e le variabili non utilizzate sono errori.

5.3 Organizzazione in package

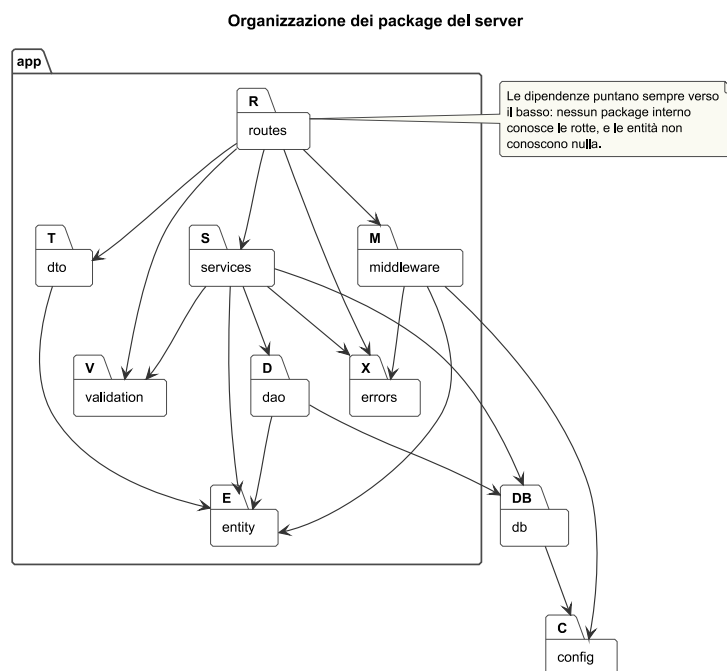


Figura 5.1. Package del server e direzione delle dipendenze.

La struttura è la stessa per tutti i sottosistemi. Le entità sono raggruppate per sottosistema, in sottocartelle di `entity`; rotte, servizi e DAO sono invece raggruppati per livello. In questo modo la decomposizione per sottosistema resta leggibile dai nomi dei file, e la struttura a livelli resta visibile nell'albero delle cartelle.

Il grafo delle dipendenze è aciclico e orientato verso il basso. Il package `entity` non dipende da nulla: le entità possono essere costruite e verificate senza alcuna infrastruttura.

5.4 Contratti delle classi principali

5.4.1 Livello di persistenza

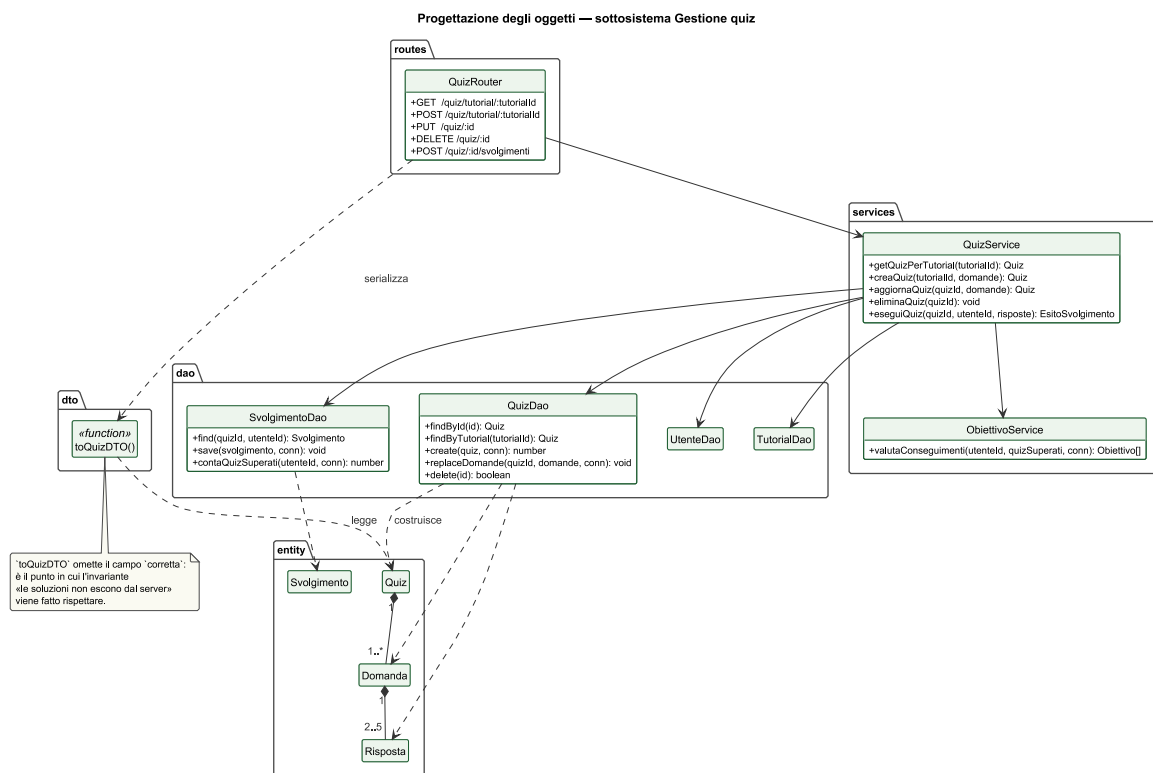


Figura 5.2. Progettazione degli oggetti del sottosistema Gestione quiz.

QuizDao è l'esempio più significativo: gestisce l'intero aggregato quiz-domande-risposte. La lettura avviene con un'unica interrogazione che unisce le tre tabelle e ricostruisce il grafo di oggetti in memoria; la scrittura accetta una connessione transazionale, perché coinvolge più tabelle e deve essere atomica.

Operazione	Contratto
<code>findById(id): Quiz null</code>	Restituisce l'aggregato completo o null. Una sola interrogazione.

Operazione	Contratto
<code>findByTutorial(tutorialId): Quiz null</code>	Come sopra, per tutorial.
<code>create(quiz, connection): number</code>	Inserisce quiz, domande e risposte assegnando gli identificativi alle entità passate. <i>Richiede</i> una connessione transazionale.
<code>replaceDomande(quizId, domande, connection)</code>	Elimina le domande esistenti — le risposte cadono in cascata — e inserisce le nuove. <i>Richiede</i> una connessione transazionale.
<code>delete(id): boolean</code>	Restituisce false se il quiz non esisteva.

5.4.2 Serializzazione

Il package `dto` contiene funzioni pure che traducono un'entità nella rappresentazione destinata al client. Nessuna rotta restituisce direttamente un'entità.

Questa regola non è una preferenza stilistica: è il meccanismo con cui due requisiti di sicurezza vengono fatti rispettare in un punto solo (DG_1, DG_2).

Funzione	Garanzia
<code>toUtenteDTO</code>	Omette l'hash della password. È l'unico modo in cui un utente lascia il server, quindi la credenziale non può sfuggire per dimenticanza in una rotta (RNF_F_1).
<code>toQuizDTO</code>	Omette il campo che distingue la risposta corretta. Rende strutturalmente impossibile erogare un quiz con le soluzioni (RNF_F_6).
<code>toTutorialDTO,</code> <code>toFeedbackDTO,</code> <code>toObiettivoDTO,</code> <code>toBadgeDTO</code>	Convertono le date in formato ISO 8601 e normalizzano i valori numerici provenienti dal DBMS.

5.4.3 Gestione degli errori

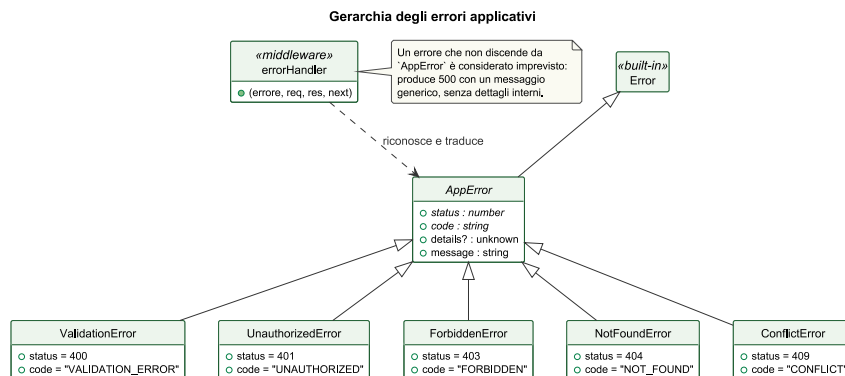


Figura 5.3. Gerarchia degli errori applicativi.

I servizi non restituiscono descrizioni dell'esito: sollevano eccezioni che portano con sé lo stato HTTP e un codice stabile. Un unico middleware le traduce in risposte. La conseguenza pratica è che una rotta si riduce a tre righe — estrarre i parametri, invocare il servizio, serializzare — e che il percorso d'errore è identico in tutto il sistema.

Un errore che non discende da `AppError` è considerato imprevisto: produce 500 con un messaggio generico, mentre il dettaglio resta nei log del server (RNF_A_2).

5.4.4 Iniezione delle dipendenze

Ogni servizio riceve i propri collaboratori come parametri del costruttore, con un valore predefinito che costruisce l'implementazione reale:

```

1 export class QuizService {
2   constructor(
3     private readonly quizDao: QuizDao = new QuizDao(),
4     private readonly svolgimentoDao: SvolgimentoDao = new SvolgimentoDao(),
5     private readonly utenteDao: UtenteDao = new UtenteDao(),
6     private readonly tutorialDao: TutorialDao = new TutorialDao(),
7     private readonly obiettivoService: ObiettivoService = new ObiettivoService(),
8   ) {}
9   ...
10 }

```

Listing 5.1. Iniezione con valore predefinito.

In esercizio la classe si costruisce da sola; nei test riceve dei doppi. Nessun contenitore di dipendenze, nessuna configurazione, e soprattutto nessun bisogno per i test di raggiungere campi privati per sostituire un collaboratore (DG_3).

Alla radice dell'applicazione la funzione `creaApp` riceve l'insieme dei servizi: è il punto in cui il grafo viene composto, ed è anche il punto in cui i test di integrazione lo sostituiscono per intero.

5.5 Pattern di progettazione adottati

5.5.1 Data Access Object

Problema. Il codice di dominio non deve conoscere SQL, e il livello di persistenza deve poter essere sostituito nei test.

Soluzione. Ogni aggregato ha un DAO che espone operazioni in termini di entità e nasconde le interrogazioni. I servizi dipendono dall'interfaccia del DAO, non dalla sua realizzazione.

Conseguenze. L'intera suite di test gira senza database (DG_3). In cambio, le interrogazioni non sono verificate dai test automatici: è il compromesso già dichiarato in sezione 1.3.

5.5.2 Facade (lato client)

Problema. Le pagine dell'interfaccia non devono conoscere URL, formato del trasporto e gestione degli errori di rete: se lo facessero, un cambiamento nell'API si propagherebbe in decine di componenti.

Soluzione. Il modulo `src/api` espone un unico oggetto organizzato per sottosistema. Le pagine invocano `api.quiz.consegna(...)` e ricevono dati già tipizzati o un errore già normalizzato [2].

Conseguenze. Il confine fra client e server è attraversato in un solo punto. Il client non importa alcun modulo del server: i tipi di trasporto sono ridichiarati in `src/types`, duplicazione accettata consapevolmente in cambio dell'indipendenza fra le due applicazioni.

5.5.3 Chain of Responsibility (middleware)

Problema. Autenticazione, autorizzazione, validazione e gestione degli errori riguardano tutte le rotte e non appartengono a nessun sottosistema.

Soluzione. Sono realizzate come middleware componibili, attraversati in sequenza: ciascuno può concludere la richiesta o passarla al successivo. Una rotta dichiara le proprie esigenze componendoli (`requireAdmin`, poi i validatori, poi il gestore).

Conseguenze. Il controllo degli accessi è dichiarativo e leggibile accanto alla rotta; la matrice di sezione 3.5 è verificabile leggendo i router. Il rovescio è che dimenticare un middleware su una rotta la lascia scoperta: per questo la matrice è coperta da test dedicati.

5.5.4 Template Method (transazioni)

Problema. Aprire una transazione, effettuare commit, gestire il rollback e rilasciare la connessione è una sequenza invariante che sarebbe ripetuta e sbagliata a ogni occorrenza.

Soluzione. La funzione `withTransaction` fissa quella sequenza e riceve come parametro il lavoro variabile da eseguire.

Conseguenze. È impossibile dimenticare un rollback o non rilasciare una connessione. La responsabilità che resta al chiamante è propagare la connessione ai DAO che invoca.

5.6 Pattern valutati e non adottati

Documentare i pattern scartati è utile quanto documentare quelli adottati: evita che la loro assenza sia scambiata per una svista.

Pattern	Applicazione ipotizzata	Perché è stato scartato
Proxy	Caricare i tutorial in forma ridotta nel catalogo e recuperare il contenuto completo solo all'apertura.	Il catalogo contiene pochi elementi e la selezione delle colonne risolve lo stesso problema con una clausola SQL. Un Proxy aggiungerebbe una classe e un livello di indirectione per un beneficio che il database offre già.
Observer	Notificare l'assegnazione di un badge a più destinatari interessati.	Esiste un solo destinatario, la risposta HTTP in corso. Il pattern avrebbe senso con notifiche via email o eventi persistenti, che sono fuori perimetro.
Strategy	Rendere sostituibile il criterio di superamento del quiz.	Il criterio è unico e fissato dal dominio. La soglia è già una costante condivisa: renderla polimorfa non risponderebbe ad alcuna necessità reale.
Repository con Unit of Work	Tracciare le modifiche alle entità e persisterle in blocco.	Le transazioni del sistema coinvolgono poche entità note a priori. <code>withTransaction</code> copre il caso d'uso senza introdurre un livello di mappatura degli oggetti.
Singleton	Garantire un'unica istanza dei servizi.	I servizi sono privi di stato: istanziarli più volte non ha conseguenze, e il Singleton renderebbe più difficile sostituirli nei test. Il solo elemento realmente unico, il pool di connessioni, è già un modulo con inizializzazione differita.

Un pattern citato in un documento di progettazione ma assente dal codice non è un dettaglio redazionale: è una descrizione errata del sistema, e induce chi legge a cercare una struttura che non esiste. Per questo i pattern di sezione 5.5 corrispondono a classi e funzioni effettivamente presenti, mentre quelli valutati e non adottati sono dichiarati come tali invece di essere taciuti.

Decisioni di progetto e razionale

Le decisioni che seguono hanno determinato la forma del sistema. Per ciascuna sono riportati il problema, la scelta, le alternative considerate e le conseguenze accettate. Sono raccolte qui perché una decisione motivata è riesaminabile, mentre una decisione implicita si può solo subire.

6.1 DP_1 — La sessione è un token firmato in un cookie httpOnly

Problema. Riconoscere l'utente fra una richiesta e l'altra proteggendo il meccanismo di riconoscimento.

Decisione. Un JWT firmato dal server, con scadenza breve, depositato in un cookie httpOnly, SameSite=Lax e Secure in produzione.

Alternative. Il token in un'intestazione Authorization, conservato dal client, sarebbe più comodo da provare con strumenti da riga di comando, ma richiede che il JavaScript di pagina possa leggerlo: una vulnerabilità di scripting basterebbe a rubarlo. Le sessioni sul server renderebbero la revoca immediata, al costo di uno stato condiviso e di un archivio di sessioni, contro DG_6.

Conseguenze. Il server resta privo di stato. In cambio, un token non può essere revocato prima della scadenza: è il motivo per cui la scadenza è breve. Il cookie viaggia cross-origin, quindi il CORS deve dichiarare un'origine esplicita e ammettere le credenziali.

6.2 DP_2 — La correzione del quiz avviene solo sul server

Problema. Impedire che l'esito di una verifica possa essere falsificato.

Decisione. La rappresentazione del quiz inviata al client non contiene l'indicazione della risposta corretta. Il client raccoglie le scelte e le consegna; il server corregge e restituisce esito e soluzioni.

Alternative. Correggere nel browser eviterebbe un giro di rete e darebbe un riscontro istantaneo, ma richiederebbe di trasmettere le soluzioni: chiunque potrebbe leggerle prima di rispondere.

Conseguenze. L'invariante è garantito dalla struttura del dato, non dalla disciplina di chi scrive il codice: la funzione di serializzazione non espone il campo, quindi non c'è modo di dimenticarsene. Il costo è una richiesta HTTP al momento della consegna.

6.3 DP_3 — Le risposte sono associate per identificativo

Problema. Correlare le scelte dell'utente alle domande del quiz.

Decisione. Il client invia coppie (`domandaId`, `rispostaId`); il server le indicizza per identificativo di domanda.

Alternative. Un elenco ordinato di identificativi di risposta, correlato per posizione all'ordine delle domande sul server, sarebbe più compatto. Introdurrebbe però un accoppiamento implicito fra l'ordine di presentazione nell'interfaccia e quello di memorizzazione: basterebbe mostrare le domande in un ordine diverso — per esempio mescolandole a ogni tentativo — perché la correzione risultasse errata senza alcun segnale.

Conseguenze. L'ordine di presentazione diventa una libera scelta dell'interfaccia. Una risposta riferita a una domanda che non appartiene al quiz viene semplicemente ignorata.

6.4 DP_4 — Il conteggio dei quiz superati è derivato

Problema. Sapere quanti quiz un utente ha superato, dato che il valore determina l'assegnazione dei badge.

Decisione. Il valore è ricalcolato contando gli svolgimenti con esito positivo, e poi memorizzato nella colonna `utente.quiz_superati` come denormalizzazione di comodo.

Alternative. Incrementare un contatore a ogni superamento è più economico, ma lega il valore alla storia delle operazioni anziché allo stato del sistema: ripetere un quiz già superato lo incrementerebbe di nuovo, e i badge finirebbero assegnati per traguardi mai raggiunti.

Conseguenze. Un'interrogazione di conteggio in più a ogni consegna, in cambio di un valore che non può divergere dalla realtà. La colonna memorizzata resta comoda per le letture e viene riscritta solo quando il conteggio effettivo differisce.

6.5 DP_5 — La valutazione media è mantenuta dal database

Problema. Mostrare nel catalogo la media dei feedback di ogni tutorial senza calcolarla a ogni lettura.

Decisione. Tre trigger su `feedback` ricalcolano `tutorial.valutazione` a ogni inserimento, modifica o cancellazione.

Alternative. Calcolarla nel servizio richiederebbe di ricordarsene in tre punti diversi, e due scritture concorrenti potrebbero lasciare un valore errato. Calcolarla a ogni lettura del catalogo costringerebbe a un raggruppamento su tutta la tabella dei feedback.

Conseguenze. Il valore è coerente per costruzione, anche di fronte a scritture concorrenti. In cambio, una parte del comportamento del sistema vive nel database, non è visibile nel codice applicativo e non è coperta dalla suite di test: è dichiarata qui e nel Test Document.

6.6 DP_6 — Quiz, domande e risposte formano un solo aggregato

Problema. Stabilire quali entità abbiano un DAO proprio.

Decisione. Un unico QuizDao governa quiz, domande e risposte. Non esistono DAO per domanda e risposta.

Alternative. Un DAO per tabella è la scelta più immediata, ma introduce tre problemi. La lettura di un quiz richiederebbe un'interrogazione per ogni domanda e una per ogni risposta. Le domande potrebbero essere scritte senza il loro quiz, in stati che il dominio non ammette. E i DAO di domanda e risposta sarebbero troppo sottili per essere verificati in modo significativo da soli, pur restando il punto in cui è più facile sbagliare l'ordine di un parametro.

Conseguenze. La lettura di un quiz è una sola interrogazione (RNF_P_1). L'aggregato è sempre coerente: non esiste un modo di scrivere una domanda senza il suo quiz.

6.7 DP_7 — Il client non condivide codice con il server

Problema. Evitare che i due lati divergano sui tipi scambiati.

Decisione. Il client richiama in `src/types` le strutture che riceve dall'API. Nessun import attraversa il confine.

Alternative. Importare le entità del server direttamente nel client eviterebbe la duplicazione ed è tecnicamente possibile con una regola di compilazione dedicata. Il client dipenderebbe però da classi che contengono logica di dominio e accesso ai dati, e non potrebbe essere costruito senza il server.

Conseguenze. I due progetti si compilano e si distribuiscono in modo indipendente. Il prezzo è una duplicazione delle definizioni di trasporto, accettata perché DTO ed entità hanno ragioni di esistere diverse e non devono cambiare insieme.

6.8 DP_8 — L'assistente conversazionale è un componente esterno

Problema. Offrire assistenza immediata a un pubblico inesperto.

Decisione. Il sistema può incorporare un widget di dialogo fornito da terzi, abilitato solo se configurato. Non esiste logica conversazionale propria.

Alternative. Realizzare un assistente interno richiederebbe competenze, infrastruttura e dati di addestramento estranei agli obiettivi del progetto; un assistente inaffidabile, su questo pubblico, sarebbe dannoso più che utile.

Conseguenze. La funzionalità è dichiarata fuori perimetro nel RAD. L'applicazione funziona identicamente senza il widget, e nessun dato dell'utente gli viene trasmesso dal sistema.

Glossario

7.1 Termini del dominio

Termine	Definizione
Utente	Persona registrata alla piattaforma. Consulta i tutorial, svolge i quiz, lascia feedback e consegue badge. Nel codice: classe <code>Utente</code> .
Amministratore	Utente con privilegi di gestione: crea e modifica tutorial, quiz e obiettivi, modera i feedback e amministra gli account. È un ruolo dell'entità <code>Utente</code> , non un'entità distinta.
Ruolo	Attributo dell'utente che ne determina i permessi. Assume i valori <code>utente</code> e <code>admin</code> . Nel codice: enumerazione <code>Ruolo</code> .
Tutorial	Unità didattica composta da titolo, immagine di copertina, contenuto testuale e categoria. È l'oggetto centrale del catalogo.
Categoria	Classificazione tematica di un tutorial. L'insieme è chiuso: Internet, Social Media, Tecnologia, Sicurezza.
Quiz	Prova di verifica associata a un solo tutorial, composta da domande a scelta multipla. La correzione è responsabilità esclusiva del server.
Domanda	Quesito a scelta multipla appartenente a un quiz, con esattamente una risposta corretta fra le opzioni proposte.
Risposta	Singola opzione di una domanda. L'indicazione di correttezza non lascia mai il server prima della consegna del quiz.
Svolgimento	Registrazione del tentativo di un utente su un quiz: esito, numero di risposte esatte e data. Un utente conserva un solo svolgimento per quiz.
Soglia di superamento	Frazione di risposte corrette necessaria a superare un quiz, pari al 70%.
Feedback	Valutazione da 1 a 5 con commento, lasciata da un utente su un tutorial. Un utente può lasciarne al più uno per tutorial.
Valutazione media	Media dei feedback ricevuti da un tutorial, mantenuta dal database e trattata come sola lettura dal codice applicativo.

Termine	Definizione
Obiettivo	Traguardo raggiungibile superando un numero prestabilito di quiz. Al raggiungimento viene assegnato il badge associato.
Badge	Riconoscimento visivo assegnato a un utente al conseguimento di un obiettivo.
Conseguimento	Associazione fra un utente e un obiettivo raggiunto, con la data di assegnazione.
Catalogo	Insieme dei tutorial pubblicati, consultabile anche senza autenticazione.
Sessione	Periodo in cui un utente è riconosciuto dal sistema, materializzato da un token firmato conservato in un cookie <code>httpOnly</code> .

7.2 Termini tecnici

Termine	Definizione
DAO	<i>Data Access Object</i> : oggetto che incapsula l'accesso a una tabella o a un aggregato del database, isolando il resto del sistema dal linguaggio SQL.
DTO	<i>Data Transfer Object</i> : struttura dati priva di comportamento usata per trasferire informazioni oltre il confine del sistema. In Tech4All è l'unico punto in cui le entità vengono serializzate verso il client.
Servizio	Oggetto che realizza le regole di dominio di un sottosistema, coordinando uno o più DAO. Non conosce il protocollo HTTP.
Middleware	Funzione interposta nella catena di elaborazione di una richiesta HTTP, che può interromperla o arricchirla.
Aggregato	Insieme di entità con un unico punto d'accesso e un ciclo di vita comune. In Tech4All quiz, domande e risposte formano un solo aggregato.
Hash	Trasformazione irreversibile di un valore. Le password sono conservate solo come hash, mai in chiaro.
Sanificazione	Rimozione dai contenuti HTML dei costrutti eseguibili, applicata prima della persistenza.
Transazione	Sequenza di operazioni sul database che viene applicata per intero o non applicata affatto.
Copertura del codice	Frazione del codice sorgente eseguita durante i test, misurata su istruzioni, dichiarazioni, funzioni e righe.

7.3 Acronimi

Sigla	Significato
API	<i>Application Programming Interface</i>
CI	<i>Continuous Integration</i>
CORS	<i>Cross-Origin Resource Sharing</i>
CRUD	<i>Create, Read, Update, Delete</i>
DBMS	<i>Database Management System</i>
DTO	<i>Data Transfer Object</i>
DAO	<i>Data Access Object</i>
FURPS+	<i>Functionality, Usability, Reliability, Performance, Supportability e vincoli aggiuntivi</i>
HTTP	<i>HyperText Transfer Protocol</i>
JSON	<i>JavaScript Object Notation</i>
JWT	<i>JSON Web Token</i>
ODD	<i>Object Design Document</i>
RAD	<i>Requirements Analysis Document</i>
REST	<i>Representational State Transfer</i>
RF	Requisito Funzionale
RNF	Requisito Non Funzionale
SDD	<i>System Design Document</i>
SQL	<i>Structured Query Language</i>
UC	<i>Use Case</i> , caso d'uso
UML	<i>Unified Modeling Language</i>
XSS	<i>Cross-Site Scripting</i>

Tracciabilità delle decisioni di progetto

Questa appendice costituisce il secondo anello della catena di tracciabilità del progetto. Il RAD lega requisiti, casi d'uso e scenari; qui i requisiti non funzionali sono legati agli obiettivi di progettazione e ai meccanismi che li realizzano; il Test Document chiude la catena legando ogni requisito ai casi di test che lo verificano.

A.1 Requisiti non funzionali, obiettivi e meccanismi

Requisito	Obiettivo	Meccanismo di realizzazione
RNF_F_1	DG_1	Hashing bcrypt in AutenticazioneService; omissione dell'hash in toUtenteDTO (sezione 5.4.2).
RNF_F_2	DG_1	Middleware requireAuth (sezione 3.5).
RNF_F_3	DG_1	Middleware requireAdmin (sezione 3.5).
RNF_F_4	DG_1	Identità presa dal token; nessuna rotta accetta un identificativo utente per stabilire l'autore (DP_1).
RNF_F_5	DG_1	Sanificazione dell'HTML in ingresso in TutorialService.
RNF_F_6	DG_1, DG_2	toQuizDTO omette il campo delle soluzioni; correzione lato server (DP_2).
RNF_F_7	DG_1	Nomi generati dal server; risoluzione e verifica del percorso nella cartella consentita.
RNF_U_1	—	Messaggi d'errore in italiano prodotti dal server; il client non ne compone di propri.
RNF_U_2	DG_2	Validatori dichiarativi sulle rotte; l'elenco dei campi non conformi è nel corpo della risposta.
RNF_U_3	—	Conferma esplicita nell'interfaccia prima delle operazioni irreversibili.
RNF_U_4	—	Rotte di lettura prive di middleware di autenticazione (sezione 3.5).

Requisito	Obiettivo	Meccanismo di realizzazione
RNF_A_1	DG_5	withTransaction attorno alle operazioni composite (sezione 3.4).
RNF_A_2	DG_1	Gestore centralizzato degli errori (sezione 5.4.3).
RNF_A_3	DG_3	Iniezione delle dipendenze (sezione 5.4.4) e soglie di copertura imposte dalla suite.
RNF_A_4	DG_5	Conteggio derivato dagli svolgimenti; uno svolgimento superato non viene sovrascritto (DP_4).
RNF_P_1	—	Aggregato quiz letto con una sola interrogazione (DP_6).
RNF_P_2	—	Filtro e ricerca eseguiti da TutorialDao in SQL.
RNF_S_1	DG_4	Stessa stratificazione in tutti i sottosistemi (sezione 3.2).
RNF_S_2	DG_2	Modulo dei vincoli di dominio condiviso fra validatori delle rotte e servizi.
RNF_S_3	DG_3	Analisi statica, controllo dei tipi, test e compilazione eseguiti a ogni integrazione.

A.2 Casi d'uso e sottosistemi

Caso d'uso	Nome	Sottosistema	Servizio
UC_01	Registrarsi	Autenticazione	AutenticazioneService
UC_02	Autenticarsi	Autenticazione	AutenticazioneService
UC_03	Terminare la sessione	Autenticazione	middleware di sessione
UC_04	Gestire il proprio profilo	Gestione account	AccountService
UC_05	Eliminare il proprio account	Gestione account	AccountService
UC_06	Consultare il catalogo	Gestione tutorial	TutorialService
UC_07	Cercare un tutorial	Gestione tutorial	TutorialService
UC_08	Consultare un tutorial	Gestione tutorial	TutorialService
UC_09	Svolgere un quiz	Gestione quiz	QuizService
UC_10	Conseguire un obiettivo	Gestione obiettivi	ObiettivoService
UC_11	Consultare i propri progressi	Gestione obiettivi	ObiettivoService
UC_12	Gestire il proprio feedback	Gestione feedback	FeedbackService
UC_13	Pubblicare un tutorial	Gestione tutorial	TutorialService

Caso d'uso	Nome	Sottosistema	Servizio
UC_14	Caricare un'immagine	Gestione tutorial	middleware di upload
UC_15	Aggiornare un tutorial	Gestione tutorial	TutorialService
UC_16	Ritirare un tutorial	Gestione tutorial	TutorialService
UC_17	Definire il quiz	Gestione quiz	QuizService
UC_18	Rimuovere il quiz	Gestione quiz	QuizService
UC_19	Gestire gli obiettivi	Gestione obiettivi	ObiettivoService
UC_20	Amministrare gli account	Gestione account	AccountService
UC_21	Moderare i feedback	Gestione feedback	FeedbackService

UC_03 e UC_14 non hanno un servizio applicativo: sono realizzati interamente da middleware. Il primo consiste nella rimozione del cookie di sessione, il secondo nella ricezione e normalizzazione di un file. In entrambi i casi non esistono regole di dominio da applicare, e interporre un servizio vuoto avrebbe aggiunto un livello senza contenuto.

Riferimenti bibliografici

- [1] Bernd Bruegge e Allen H. Dutoit. *Object-Oriented Software Engineering: Using UML, Patterns, and Java*. 3^a ed. Upper Saddle River, NJ: Pearson, 2010.
- [2] Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA: Addison-Wesley, 1994.
- [3] Michael Jones, John Bradley e Nat Sakimura. *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force. 2015. DOI: 10.17487/RFC7519.
- [4] OWASP Foundation. *OWASP Top 10: 2021*. <https://owasp.org/Top10/>. 2021. (Visitato il giorno 27/08/2026).
- [5] Niels Provos e David Mazières. *A Future-Adaptable Password Scheme*. Proceedings of the USENIX Annual Technical Conference. 1999.